```
#!/usr/bin/env python3
"""Extract page-level text from a directory of PDF textbooks using PyMuPDF.

Each record matches the manifest schema:
    {"book_id": str, "title": str, "page": int (1-indexed), "text": str}

Usage:
    python extract_textbooks.py ./textbooks -o pages.jsonl
    python extract_textbooks.py ./textbooks -o pages.jsonl --recursive --keep-emp

Requires: pip install pymupdf
"""

import argparse
import hashlib
import json
import logging
import re
import sys
from pathlib import Path
from typing import Iterator

import fitz  # PyMuPDF

log = logging.getLogger("textbook_extract")

# Titles that PDF authoring tools often leave behind; treat as missing.
JUNK_TITLE = re.compile(r"^(untitled|microsoft word\b.*|.*\.(docx?|indd|tex|pdf))


def slugify(text: str) -> str:
    return re.sub(r"[^a-z0-9]+", "-", text.lower()).strip("-")


def make_book_id(path: Path, root: Path, used: set) -> str:
    """Stable, readable ID from the filename; a short path hash resolves collisio
    slug = slugify(path.stem)
    if not slug:
        slug = "book"
    if slug in used:
        digest = hashlib.sha1(str(path.relative_to(root)).encode()).hexdigest()[:
        slug = f"{slug}-{digest}"
    used.add(slug)
    return slug
```

```
def resolve_title(doc: fitz.Document, path: Path) -> str:
    """Use PDF metadata title; fall back to a cleaned-up filename."""
    try:
        title = (doc.metadata or {}).get("title") or ""
    except Exception as exc:  # corrupt metadata dictionary
        log.warning("%s: could not read metadata (%s)", path.name, exc)
        title = ""

    title = title.strip()
    if not title or JUNK_TITLE.match(title):
        fallback = re.sub(r"[_\-]+", " ", path.stem).strip()
        log.warning("%s: missing/unusable title metadata, using '%s'", path.name,
        return fallback
    return title


def extract_book(path: Path, book_id: str, keep_empty: bool = False) -> Iterator[
    """Yield one manifest record per page. Never raises on per-page failures."""
    try:
        doc = fitz.open(path)
    except Exception as exc:
        log.error("%s: cannot open (%s); skipping file", path.name, exc)
        return

    with doc:
        if doc.needs_pass and not doc.authenticate(""):
            log.error("%s: password-protected; skipping file", path.name)
            return

        title = resolve_title(doc, path)
        failed, empty = 0, 0

        for index in range(doc.page_count):
            page_no = index + 1
            try:
                page = doc.load_page(index)
                text = page.get_text("text", sort=True).strip()
            except Exception as exc:
                failed += 1
                log.warning("%s p.%d: extraction failed (%s)", path.name, page_no
                if keep_empty:
                    yield {"book_id": book_id, "title": title, "page": page_no, "
                continue

            if not text:
```

```
                empty += 1  # likely a scanned/image-only or blank page
                if not keep_empty:
                    continue

            yield {"book_id": book_id, "title": title, "page": page_no, "text": t

        if failed or empty:
            log.info("%s: %d failed page(s), %d empty page(s) of %d",
                     path.name, failed, empty, doc.page_count)


def extract_directory(root: Path, recursive: bool = False, keep_empty: bool = Fal
    pattern = "**/*" if recursive else "*"
    pdfs = sorted(p for p in root.glob(pattern) if p.is_file() and p.suffix.lower
    if not pdfs:
        log.warning("No PDF files found in %s", root)
        return

    used_ids: set = set()
    for path in pdfs:
        book_id = make_book_id(path, root, used_ids)
        log.info("Processing %s -> book_id=%s", path.name, book_id)
        yield from extract_book(path, book_id, keep_empty)


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__, formatter_class=argpars
    parser.add_argument("directory", type=Path, help="Directory containing PDF te
    parser.add_argument("-o", "--output", type=Path, default=Path("pages.jsonl"),
    parser.add_argument("-r", "--recursive", action="store_true", help="Search su
    parser.add_argument("--keep-empty", action="store_true",
                        help="Emit records with empty text for blank/failed pages
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()

    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO,
                        format="%(levelname)s %(message)s")

    if not args.directory.is_dir():
        log.error("Not a directory: %s", args.directory)
        return 1

    count = 0
    with args.output.open("w", encoding="utf-8") as fh:
        for record in extract_directory(args.directory, args.recursive, args.keep
            fh.write(json.dumps(record, ensure_ascii=False) + "\n")
            count += 1
```

```
    log.info("Wrote %d page records to %s", count, args.output)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```